Encode the thought - neural network training for understanding semantic instead of predicting tokens

“Thoughts die the moment they are embodied by words.” A. Schopenhauer

Lyubimkov Dmitri

E-mail: loftlong@gmail.com

2026

Abstract

Modern large language models effectively predict tokens, but often substitute statistical syntax for understanding meaning. Building on the premise that thought is encoded at the level of sentences and semantic fragments, rather than individual words, the Encode_thought project explores the architecture of explicit semantic compression of text into a compact latent representation. Versions V1 and V2 empirically confirmed the performance of the slot bottleneck in the Teacher Forcing mode but revealed the fundamental instability of closed-loop autoregression due to exposure bias and prefix attention blurring. This work addresses this issue by switching to native cross-attention: a frozen T5-small decoder is continuously conditioned on a matrix of slots (16×192) serving as encoder_outputs. Adaptation of the attention matrices via LoRA, curriculum embedding noise, and slot-alignment loss ensures geometric consistency of the latent space and training of a model to compensate for its own drift. The architecture demonstrates stable autoregressive generation on 80–128 tokens without lexical collapse, preserving syntax, style, and causal relationships with a Theme Overlap of ~0.72 and a fair match of train/val metrics. Residual mutations of names and objects are identified as the capacity limit of a fixed bottleneck. As a next step, a transition from a single static matrix to a sequence of interconnected semantic blocks with inter-fragment causal attention is justified, which removes the limit on text length and eliminates gaps at sentence boundaries. The pipeline is fixed as a reproducible baseline for semantic generation control tasks and is ready for scaling to long narratives.

Table of contents

1. Encode thought Версии 1 и 2

The first version of Encode Thought naively considered the idea and necessity of encoding the semantics/meaning of text instead of words/tokens. It drew an analogy and distinction between words and thoughts and raised the issue of the lack of explicit thought identification in modern LLMs. As an alternative, it proposed encoding not individual words/tokens but semantically complete constructs, such as sentences. A possible mechanism for such encoding was the use of a compact bottleneck encoder-decoder pair.

Version 1 remained theoretical—it articulated the idea but lacked any architectural choices or experimental verification. The next paper (Encode_thoughtV2) was intended to fill this gap. The full text of Version 1 can be found at https://github.com/loftyara/Encode_thought/ or https://loftyara.github.io/encode_thought.html

Version 2 moved the concept from the theoretical realm to the experimental. An architecture was developed and trained that combines a Set Transformer for order-invariant encoding, a learnable query engine with cross-attention for extracting a compact matrix of semantic slots, and a transformer decoder for text recovery. Experiments on the TinyStories dataset using frozen base encoders (in a minimal configuration of ~0.9 million trainable parameters) empirically confirmed the main hypothesis: the model successfully extracts an invariant semantic core and recovers 80–90% of the original text in Teacher Forcing mode. This demonstrated that the semantics of simple narratives has low effective dimensionality and can be reliably compressed into a continuous slot representation without losing plot structure and causal relationships.

However, testing in a closed autoregressive loop (AR and Raw AR) revealed a fundamental limitation: generation collapsed into repetitive lexical patterns after just 5–10 steps. Diagnostics revealed that the cause lay not in a lack of capacity or architectural errors, but in exposure bias—the model was trained on an ideal context distribution and lacked a mechanism to compensate for the cascading drift of its own predictions. Increasing the number of parameters or training epochs did not eliminate this gap. Thus, Encode_thoughtV2 successfully completed the first stage of the study, proving the feasibility of slot-based compression and reconstruction, but clearly identified the limits of applicability: without stabilizing the autoregressive loop, the practical value of the concept remains limited. This paper continues the research, focusing exclusively on closing the AR loop and transitioning from parallel training to robust character-by-character generation.

This work (Encode_thoughtV3) aims to address these shortcomings. The full text of Version 2 can be found at https://github.com/loftyara/Encode_thoughtV2/ or https://loftyara.github.io/encode_thoughttv2.html

2. Goals and objectives

This work continues the research begun in Encode_thoughtV2 and aims to achieve robust autoregressive text generation from compressed semantic representations (slots). While the previous version empirically demonstrated the feasibility of slot compression and reconstruction in Teacher Forcing mode, it also revealed the fundamental instability of the closed AR loop due to exposure bias. The current stage focuses exclusively on closing the generation loop and transitioning from parallel training to symbol-by-symbol reconstruction without distribution collapse.

The main objectives of the project:

- Diagnosis of the causes of autoregressive collapse and analysis of the gap between the distributions of Teacher Forcing and closed-loop generation
- Transition from prefix-conditioning to an architecture with native cross-attention (T5-small) for continuous conditioning of the decoder on semantic slots at each generation step
- Adaptation of cross-attention matrices via LoRA (q, k, v, o) without catastrophic forgetting of the linguistic priors of the base model
- Scaling the capacity of the latent space (16 slots × 192 dim) and implementing Slot-Alignment Loss to explicitly retain the semantic core of the target text in the decoder space
- Implementation of curriculum embedding noise and gradient flow stabilization to train a model to compensate for its own prediction drift

- Formation of a correct validation pipeline (fair train/val split, early stopping by generalization, single checkpoint) and AR-robustness assessment metrics (Theme Overlap, length of coherent fragment, absence of lexical attractors)
- Bring optimization to a fair plateau, commit to an interim release, and document architectural limits of applicability for planning the next iteration

3. Selection of basic transformer encoders

Within Encode_thoughtV2, a comparative evaluation of several compact encoder-only models (distilbert-base, bert-mini, MiniLM, TinyBERT, and jina-small) was conducted. The results showed that for the task of extracting an invariant semantic core from short narratives, the critical factors are not the encyclopedic capacity of the encoder, but the stability of syntactic parsing, low latent dimension, and gradient predictability. Based on these findings, the experimental encoder evaluation was discontinued in the current version. The prajjwal1/bert-mini model (256 latent dimension, 4 layers, ~11M parameters, pure MLM pre-training strategy) was selected as the fixed base encoder. The model is loaded with weights from Hugging Face Hub and used in frozen mode exclusively to transform a sequence of tokens into a sequence of embeddings. Further meaning extraction, slot compression, and autoregressive reconstruction are performed by the EncodeThought target architecture. Eliminating the need to test alternative encoders allows computing resources to be focused on AR loop stabilization and cross-attention adaptation, without blurring the experimental focus.

4. Selecting a neural network architecture

When a child begins to understand speech, he or she is able to immediately extract meaning from it, without relying on explicit grammatical rules or extensive knowledge bases. This observation suggests that semantic extraction is not a computationally complex task and should not require gigantic architectures or multi-stage pipelines. The text already contains all the necessary information for reconstructing thought; the task of the neural network is not to "create" meaning, but to filter out syntactic noise and compress the invariant kernel into a compact representation.

In V2, we empirically confirmed that such compression is feasible: even a minimal configuration successfully recovered 80–90% of the original text in Teacher Forcing mode. However, experiments also clearly demonstrated the limits of this approach's applicability: a standard transformer decoder, trained on an ideal context, is unable to maintain semantic support in a closed autoregressive loop. The problem turned out to be not a lack of capacity, but a problem with the conditioning mechanism—prefix injection of slots quickly "dissolved" in the growing context, and the model lost its connection with the original meaning.

The current version of the architecture is built on a simple but fundamental principle: if the base language model already masters syntax and style, the target network should focus exclusively on meaning management, rather than generating it from scratch. Instead of training our own decoder, we use a frozen seq2seq model (T5-small) with a native cross-attention mechanism. Slots are no longer a static prefix; they act as encoder_outputs, accessed by the decoder at each generation step. This eliminates the problem of attention blurring and guarantees continuous semantic binding.

Why do we think this approach will work?

First, there's the separation of concerns. Language priors remain in the frozen weights of the base model, and the trainable parameters (~150K) are concentrated exclusively in the slot projector and LoRA cross-attention adapters. This eliminates catastrophic forgetting, stabilizes gradients, and allows the network to focus solely on semantic alignment.

Second, geometric consistency. The introduction of Slot-Alignment Loss explicitly aligns the average slot representation with the semantic core of the target text in the decoder space. This prevents latent code drift into abstract or lexically attractive but semantically empty states.

Third, learning from its own trajectories. Curriculum embedding noise gradually replaces the ideal context with the model's predictions during the training phase. The system learns to compensate for cascade drift before encountering it in the inference, which directly closes the autoregressive loop.

Thus, the architecture has evolved from "compression and parallel reconstruction" to "continuous semantic steering." We don't attempt to force the compact network to relearn language. We use a pre-existing language engine and train only the rudder—a slot matrix that maintains the original thought's direction, regardless of the length of the generated sequence and accumulated prediction errors.

4.1. Statement of the problem and requirements for presentation

The goal of the architecture remains the extraction of an invariant semantic core from the text—a compact representation that preserves meaning, narrative structure, and causal relationships, but abstracts away syntactic form and lexical design. Experiments with version V2 empirically confirmed that such a core exists and has a low effective dimensionality: even the minimal configuration successfully recovered 80–90% of the source text in Teacher Forcing mode. However, the transition to closed-loop autoregression revealed a fundamental limitation: a one-time injection of slots at the beginning of the context is unable to maintain semantic support throughout the entire generation. The decoder's attention inevitably erodes, and the model loses its connection to the original meaning.

Based on these findings, the requirements for the target representation and the mechanism for its use were significantly revised. Meaning is no longer viewed as a static prefix; it must act as a continuous semantic anchor that the decoder consults at every generation step. The following requirements were formulated:

1. Continuous conditioning. Slots should not "dissolve" in the growing context. The architecture must provide an explicit mechanism for accessing the semantic core at each step of the autoregression (native cross-attention), eliminating the dependence of the generation stability on the sequence length.
2. Resilience to cascade drift (AR stability). The representation should allow the model to compensate for its own prediction errors. Training should occur not only on the ideal context but also on trajectories containing the model's predictions, to close the autoregressive loop before the inference stage.
3. Geometric alignment with the decoder space. The latent slot space and the embedding space of the underlying language model must be explicitly aligned. Without forced alignment of the semantic core with the target distribution, the decoder will rely on its own linguistic priors rather than on the encoded meaning.
4. Scalable vector matrix. A single vector remains an unacceptable "bottleneck." Meaning is encoded by a set of N slots of dimension D, where the representational capacity adapts to the complexity of the narrative. Slot specialization (entities, actions, relationships) arises naturally under the pressure of the loss function, without explicit discrete rules or iterative competition.
5. 5. End-to-end learning with shared responsibility. Semantic extraction and text generation must be trained in a coordinated manner, but without catastrophic loss of language skills. The base language model preserves syntax and style in frozen weights, and the trained parameters focus exclusively on the slot projector and cross-attention adaptation, ensuring stable gradients and predictable convergence.

These requirements shape the architectural contract of the current version: meaning is no longer "embedded" in context once, but instead continuously controls generation through a dedicated attentional channel. This shifts the focus from parallel reconstruction to robust symbol-by-symbol generation, a prerequisite for the practical applicability of the concept.

4.2. 4.2. Analysis of tested approaches to autoregressive stabilization

After empirically confirming the feasibility of slot compression in V2, the instability of closed-loop autoregression emerged as a key obstacle. A model trained on the ideal context distribution (Teacher Forcing)

collapsed into lexical loops after just 5–10 generation steps. Further experiments focused exclusively on finding a mechanism capable of maintaining semantic support throughout the entire AR trajectory without destroying the gradient flow or generalization metrics. During the development process, three fundamentally different approaches to decoder conditioning were tested.

1. Prefix conditioning (GPT-2) with soft and hard anchoring

Slots were projected into the embedding space of the base model and added to the beginning of the context as a static prefix. To compensate for cascading drift, embedding interpolation (1-α) GT + α Pred (soft anchoring) and periodic context pruning with slot reinjection (hard anchoring) were used. Diagnostics showed that attention to the prefix inevitably erodes as the sequence grows (~1/N). Soft anchoring disrupts positional geometry and attention distribution, causing smooth semantic drift without stabilizing entities. Hard context pruning eliminates syntactic collapse but does not preserve causal relationships and names. This approach did not address the fundamental problem of attention dilution and was rejected.

2. Discrete AR-Rollout with Detached Context

Training switched from Teacher Forcing to an 8-step autoregressive model, where context was updated via discrete sampling (detached), and gradients were passed only through logits. The goal was to teach the model to compensate for its own prediction errors. In practice, the val loss exploded when switching, as optimization shifted from the TF manifold to the distribution of its own trajectories, rendering standard validation pointless. Gradients became noisy, epoch times increased by a factor of 5-6 due to a sequential Python loop without a KV cache, and early stopping broke down due to microfluctuations. This approach proved incompatible with stable training, fair generalization assessment, and limited VRAM.

3. Native Cross-Attention with LoRA adaptation (T5-small)

Slots were repurposed as encoder_outputs in the seq2seq architecture, ensuring continuous access by the decoder to the semantic core at each generation step. Cross-attention matrices (q, k, v, o) were adapted via LoRA (r=8) without unfreezing the underlying weights, eliminating the risk of catastrophic forgetting of linguistic priors. Combined with curriculum embedding noise (α≤0.12) and Slot-Alignment Loss, this approach eliminated the dilution problem architecturally, preserved differentiability and gradient stability, ensured fast training (~20–25 sec/epoch), and correct operation of the val loss/early stopping algorithm. The autoregressive loop was successfully closed, syntactic collapse was eliminated, and Theme Overlap reached a stable plateau of ~0.72 with a fair match between train/val metrics.

4. Diagnostic conclusions

Comparative experiments clearly demonstrated that the robustness of AR generation is determined not by the size of the trained parameters or the duration of training, but by the decoder's access mechanism to the semantic representation. Prefix methods and discrete rollouts create a structural gap between the training and inference distributions, leading to either attention blurring or gradient instability. Native cross-attention with adaptive tuning closes this gap, ensuring continuous semantic control without disrupting the model's linguistic base. This approach was finalized and formed the basis of the architecture described in detail in Section 4.3.

4.3. Selected architecture

The resulting architecture is a hybrid of a Set Transformer, a learnable slot mechanism, and a frozen seq2seq model, combining the mathematical rigor of set processing with the robustness of native cross-attention:

Текст → Token Embeddings → Set Transformer Encoder → Hidden States

↓

Learnable Queries → Cross-Attention → Slot Matrix (16 × 192)

↓

SlotCrossProjector (192 → 512) → T5 Decoder (frozen) + Cross-Attention LoRA

↓

Continuous autoregressive generation

4.3.1. Encoder: Set Transformer with ISAB blocks

The Set Transformer, specifically designed for processing set data, remains the encoder. Experiments in V2 empirically confirmed the robustness and gradient stability of this block, so it remains unchanged in the current version. Key features:

- Order invariance. The architecture is constructed independently of permutations of input elements, eliminating the need to memorize syntactic patterns associated with token positions.
- Efficiency. Using Induced Set Attention Blocks (ISAB) provides linear O(n) complexity in sequence length, which is critical for scaling and maintaining constant VRAM consumption.
- Stability. The attention mechanisms in Set Transformer are mathematically driven and do not contain iterative or competitive processes, ensuring predictable convergence.

4.3.2. Slot Mechanism and Projector: Learnable Queries + Cross-Attention + Linear Projection

Extraction of semantic slots is implemented through a set of trainable query vectors that independently aggregate information from the encoder outputs. The representation capacity is scaled to 16 slots of dimension 192, providing sufficient capacity to retain the plot skeleton, entities, and causal relationships.

- No contention. Each query interacts with the encoder via standard cross-attention without any mechanisms to suppress other slots. This ensures smooth gradients and stable convergence.
- Natural specialization. The division of responsibilities between slots arises under the pressure of the reconstruction loss function and Slot-Alignment Loss: duplicating information is disadvantageous, as it increases the error. Slots independently distribute aspects of meaning during the optimization process.
- • Projection into decoder space. The new SlotCrossProjector component (Linear → LayerNorm → GELU) explicitly aligns the slot geometry with the latent space of the underlying language model (512D). This eliminates the distributional gap and ensures that the decoder's cross-attention works.

4.3.3. Decoder: T5-small with native Cross-Attention and LoRA adaptation

Instead of training a custom transformer decoder, a frozen T5-small seq2seq model (~60M parameters) is used. Slots are not added as a static prefix, but are injected as encoder_outputs, which activates the native cross-attention mechanism at each autoregressive step.

- Continuous conditioning. The decoder accesses the semantic core at each generation step, completely eliminating the attention dilution problem typical for prefix methods of GPT-like architectures.
- LoRA cross-attention adaptation. The q, k, v, and o matrices in the cross-attention layers are adapted via LoRA (r=8, α=16). This allows the decoder to dynamically focus on slots while preserving language priors in frozen weights and preventing catastrophic forgetting.
- • Separation of concerns and AR robustness. The base model is responsible for syntax, style, and grammar, and the trainable parameters (~150K) focus exclusively on the slot projector and attention adapters. Curriculum embedding noise (α≤0.12) trains the model to compensate for cascading drift in its

own predictions, and Slot-Alignment Loss geometrically aligns the average slot representation with the semantic core of the target text.

The decoder accepts the slot matrix and reconstructs the embedding sequence of the base model. The subsequent transition to tokens occurs during the analysis phase. Using a standard transformer decoder ensures:

- Compatibility with modern generation practices.
- Possibility of cross-lingual recovery (encoding in one language, decoding in another), which clearly forces the model to highlight the invariant meaning.

The selected architecture satisfies all the requirements formulated in section 4.1:

#	Requirement	Implementation in the current version
1	Continuous conditioning	Slots are injected as T5 encoder_outputs. Native cross-attention accesses them at every step of AR generation.
2	AR stability	Curriculum embedding noise + LoRA cross-attention + frozen language priors. The model is trained on its own drift trajectories.
3	Geometric consistency	SlotCrossProjector (192→512) + Slot-Alignment Loss (cosine). The slot latent space is explicitly aligned with the decoder distribution.
4	Scalable vector matrix	16 slots x 192 dim. Capacity is adapted to the complexity of narratives; specialization arises through Loss without discrete rules.
5	End-to-end with separation of concerns	Direct learning from embeddings. Syntax is frozen in T5, meaning is learned in the projector and LoRA adapters (~150K parameters).

Table 1. Compliance of the selected architecture with the requirements

5. Selecting a dataset

The TinyStories dataset was chosen to experimentally test the hypothesis regarding the feasibility of extracting an invariant semantic core from text. The project's goal is to test whether a compact architecture can extract meaning from text without relying on encyclopedic knowledge or complex linguistic constructs. TinyStories is ideal for this purpose: the dataset consists of short stories (50–300 words) written in simple language at the preschool level. The stories contain a clear cause-and-effect structure, explicit entities and actions, but minimize syntactic complexity and ambiguity. This allows us to isolate the task of semantic compression from the tasks of world understanding or lexical ambiguity resolution. The dataset consists of two text files, train and val, with stories ending with the <|endoftext|> tag.

6. 6. Preparing for launch

6.1. Launch Platform

All experiments were conducted on the author's PC with the following characteristics:

- OS Windows 11
- 32 GB DRAM
- Nvidia RTX 5070 Ti graphics card with 16 GB of memory (CUDA support)
- Python version 3.12

6.2. Loading a project

The Encode_thoughtV3 project is hosted in an open repository on GitHub. To get it to your computer, follow these steps:

6.2.1. Open the command prompt (cmd)

6.2.2. Navigate to the directory where you want to place the project (for example, D:\Projects with the command cd D:\Projects)

6.2.3. Run the git clone command https://github.com/loftyara/Encode_thoughtV3.git

6.2.4. Instead of the command line, you can use the GitHub Desktop program

6.2.5. Once completed, you'll see a new folder called Encode_thoughtV3. The entire project structure will be contained within it.

6.3. Preparing the virtual environment

To isolate project dependencies, it's recommended to use a Python virtual environment (venv). Navigate to the downloaded project directory and run the following command:

python -m venv venv

Activate the virtual environment:

venv\Scripts\activate

install pytorch:

pip install torch --index-url https://download.pytorch.org/whl/cu130

install dependencies:

pip install -r requirements.txt

6.4. Project files and directories

The list of directories is given in Table 2, the list of files in Table 3.

#	Directory	Description
1	checkpoints	saving models and/or their checkpoints
2	data	dataset
3	data/raw	raw data of the dataset
4	data/processed	calculated embeddings
5	docs	documentation
4	scripts	data preparation programs
7	src	programs for training models and analyzing results

Table 2. List of project directories

#	File	Directory	Description
1	best.pt	checkpoints	A model based on prajjwal1/bert-mini. Calculated during runtime.
2	train.txt	data/raw	Dataset, training portion. Loaded during operation.
3	val.txt	data/raw	Dataset, validation part. Loaded during runtime.
4	embeddings_all-MiniLM-L6-v2_train_chunk_nnnn.pt	data/ processed	All-MiniLM-L6-v2 embeddings. Calculated during runtime.
5	embeddings_bert-mini_train_chunk_nnnn.pt	data/ processed	prajjwal1/bert-mini embeddings. Calculated during runtime.
6	embeddings_distilbert-base-uncased_train_chunk_nnnn.pt	data/ processed	distilbert-base-uncased embeddings. Calculated during runtime.
7	embeddings_jina-embeddings-v2-small-en_train_chunk_nnnn.pt	data/ processed	Embeddings jina-embeddings-v2-small-en. Calculated during runtime.
8	embeddings_TinyBERT_General_4L_312D_train_chunk_nnnn.pt	data/ processed	TinyBERT_General_4L_312D embeddings. Calculated during runtime.

9	encode_thought.pdf	Docs	documentation
10	01_download_dataset.py	scripts	dataset loading program
11	02_gen_embeddings_bertmini.py	scripts	program for calculating embeddings all-MiniLM-L6-v2.
12	02_gen_embeddings_distilbert.py	scripts	program for calculating embeddings prajjwal1/bert-mini.
13	02_gen_embeddings_jina.py	scripts	program for calculating embeddings distilbert-base-uncased.
14	02_gen_embeddings_minilm.py	scripts	program for calculating embeddings jina-embeddings-v2-small-en.
15	02_gen_embeddings_tinybert.py	scripts	program for calculating embeddings TinyBERT_General_4L_312D.
16	dataset.py	src	library for accessing the dataset (embeddings)
17	model.py	src	library for calculating the semantic model
18	train.py	src	model training program for embeddings prajjwal1/bert-mini.
19	analyze.py	src	model analysis for embeddings prajjwal1/bert-mini.

Table 3. List of project files

7. Preparing the dataset

Activate the virtual environment:

venv\Scripts\activate

Go to the scripts directory:

cd scripts

Run the program:

python 01_download_dataset.py

Run the programs one by one:

python 02_gen_embeddings_bertmini.py

python 02_gen_embeddings_distilbert.py

python 02_gen_embeddings_jina.py

python 02_gen_embeddings_minilm.py

python 02_gen_embeddings_tinybert.py

8. Model training

Activate the virtual environment:

venv\Scripts\activate

Go to the src directory:

cd src

Run the program:

python train.py

Experiments have so far been limited to the basic prajjwal1/bert-mini transform encoder.

9. Analysis of results

10. Activate the virtual environment:

    venv\Scripts\activate

    Go to the src directory:

    cd src

    Run the program:

    python analyze.py

    Experiments have so far been limited to the basic prajjwal1/bert-mini transform encoder.

11. Parameters of the Encode_thoughtV3 model

11.1. Model Independence and Unified Pipeline

The EncodeThought architecture is intentionally designed as a model-agnostic semantic extraction module. It has no rigid constraints on the dimensionality, number of layers, or internal structure of the underlying text encoder. The input tensor (B, T, d_in) first passes through a linear projection input_proj, which adapts the arbitrary dimensionality of the underlying model embeddings (d_in ∈ [256, 768]) to a fixed internal slot space (dim_model=192). The learnable query engine aggregates the information into a fixed-size matrix (16 × 192), completely abstracting from the length of the input sequence and the order of the tokens.

SlotCrossProjector is used to interact with the generative component, explicitly aligning the slot geometry with the latent space of the target language model (512D for T5-small). The decoder accepts the slot projection as encoder_outputs and uses native cross-attention at each autoregressive step, ensuring continuous semantic conditioning without being tied to a specific generation architecture or context length.

The pipeline remains universal: replacing the base encoder or language model requires only adjusting the dimensions of the input and output projectors. The core meaning extraction engine (Set Transformer + Slot Bottleneck) and the attention adaptation mechanism (LoRA for cross-attention) remain unchanged, allowing the system to scale to more powerful base models without redesigning the architecture.

11.2. Final configuration

The final architecture parameters are shown in Table 4. The configuration is optimized to balance semantic capacity, autoregressive stability, and training efficiency on a consumer GPU.

#	Parameter	Value	Role
1	NUM_SLOTS	16	Number of trainable slot vectors (semantic representation capacity)
2	DIM_MODEL	192	Internal dimension of the Set encoder and slot bottleneck
3	NUM_HEADS	1	Attention heads in ISAB and Cross-Attention (eliminates gradient sparsification)
4	NUM_ENCODER_LAYERS	1	ISAB layers in the Set encoder
5	NUM_INDS	4	Induced elements to reduce attention complexity to O(T)
6	MAX_SEQ_LEN	128	Target sequence length and positional embedding limit
7	LORA_R / LORA_ALPHA	8 / 16	Rank and scale of LoRA adapters for cross-attention matrices (q, k, v, o)
8	MAX_NOISE_ALPHA	0.12	Maximum curriculum weight embedding noise for drift compensation training

9	SLOT_ALIGNMENT_WEIGHT	0.5	Slot-Alignment Loss (cosine) weight, which ties the slot geometry to the target text
10	EARLY_STOP_PATIENCE	8	Tolerance of plateau val loss for automatic training stop

Table 4. Final parameters of the model

The final volume of trainable parameters is ~150,000 (linear slot projector + LoRA cross-attention adapters). For comparison, the prajjwal1/bert-mini baseline encoder contains ~11 million parameters, while the t5-small frozen decoder contains ~60 million. The target architecture performs invariant meaning extraction, geometric alignment, and attention adaptation using less than 0.3% of the total computational weight of the baseline components. This empirically confirms the original hypothesis: semantic control does not require retraining the language model and can be implemented through a compact, specialized adapter layer that preserves syntax and style priors in frozen weights.

11.3. Loss Function and Learning Strategy

Instead of the classic MSE or isolated CrossEntropy, a combined loss function adapted to the problem of continuous semantic conditioning and autoregressive stabilization is used: $L = L_{CE} + \lambda L_{align}$

- CrossEntropy Loss (with padding token masking) is responsible for accurately predicting target tokens in the frozen T5 decoder space. It ensures that the syntactic correctness, grammar, and style priors of the base model are preserved without requiring their retraining.
- Slot-Alignment Loss (cosine distance) explicitly aligns the average slot projection representation with the semantic core of the target sequence in the decoder's latent space (512D). It acts as a geometric regulator that prevents slots from drifting into abstract or lexically attractive but semantically empty states. The regularization weight is fixed at $\lambda = 0.5$, which ensures a balance between token accuracy and latent code robustness.

Training is performed in Parallel Teacher Forcing mode, but the Curriculum Embedding Noise strategy is used to close the autoregressive loop. Starting from the third epoch, the ideal decoder context is gradually interpolated with the model's own predictions (maximum noise weight $\alpha = 0.12$). This allows the system to learn to compensate for cascade drift already during the training phase, without destroying parallelism or requiring unstable discrete rollouts.

The optimization pipeline is stabilized using AMP (automatic accuracy blending), gradient accumulation (2 steps), and gradient norm clipping (1.0). Early stopping (patience=8) is triggered automatically when the validation loss plateaus, ensuring that the checkpoint remains exactly at the peak of generalization. The underlying language model remains frozen throughout the entire cycle; only the slot projector and LoRA cross-attention adapters (~150K parameters) are trained, preventing catastrophic forgetting and ensuring predictable convergence.

11.4. Key engineering decisions

Training stability and reproducibility of results on a consumer GPU were ensured not by increasing model capacity, but through rigorous optimization of the computational pipeline and gradient flow management. The current version implements the following engineering solutions:

- AMP and gradient accumulation. Automatic accuracy blending (torch.amp.autocast) combined with 2-step gradient accumulation and norm clipping (1.0) provides stable optimization with ~150K trainable parameters, eliminating gradient explosions and reducing VRAM consumption to 3.5–4.0 GB.
- Consistent LoRA initialization. Cross-attention adapters are wrapped via PEFT before calling load_state_dict. This ensures precise matching of checkpoint keys, eliminates weight desynchronization during restarts, and allows for correct saving/loading of model state without manual mapping.
- Curriculum Embedding Noise. Smooth interpolation of the ideal context with the model's own predictions (α: 0→0.12) replaces unstable discrete rollouts. The system learns to compensate for cascaded drift in parallel, preserving differentiability and without destroying the TF manifold.

- Unified saving and early stopping. The pipeline saves exactly one best.pt file, overwriting it only when the validation loss improves. Early stopping with an 8-epoch tolerance automatically sets the checkpoint at the peak of generalization, preventing artifact accumulation and overfitting at plateaus.
- Adaptive data loader. The dataset is loaded from a flat chunk structure (embeddings_{model}_{split}_chunk_*.pt) with automatic dict format normalization. This eliminates path errors, speeds up tensor preparation, and allows scaling the data volume without recalculating embeddings.
- Determinism and memory management. A fixed seed (42) for torch/numpy, disabling garbage collection during epochs, and using pinned RAM tensors ensure fully reproducible results, no VRAM micro-freezes, and stable epoch times (~20–25 s on 20k samples).

These solutions form a reproducible and resource-efficient pipeline that allows optimization to reach a fair plateau without hardware limitations or gradient instability.

11.5. Limits of applicability and diagnostics

Experiments in the current version clearly divided the capabilities of the architecture into two levels: the achieved optimization ceiling and the fundamental limitations that require a change in the conditioning paradigm.

1. Closed AR loop and syntactic stability. The model reliably generates text in autoregressive mode over 80–128 tokens. Lexical collapses, cyclic patterns, and syntactic breaks are completely eliminated. The style, punctuation, and causal relationships of narratives are preserved without degradation.
2. Fair generalization. The Train and Val Theme Overlap metrics match at ~0.72. There are no discrepancies between the training and validation distributions, early stopping works correctly, and there is no overfitting. This confirms that curriculum noise and slot alignment have successfully aligned the distributions of Teacher Forcing and closed-loop generation.
3. Semantic drift as a new limit. Despite structural stability, the model exhibits localized mutations of names and objects (Roxy→Shelley, cherry→pine, cow→dog), as well as slight echoes at sentence boundaries. Diagnostics show that the cause is not architectural errors or a lack of epochs, but rather the capacitive ceiling of the current latent code (16 slots × 192 dim) and frozen T5-small linguistic priors. The slots successfully retain the plot skeleton and global semantics, but do not capture token-level entities without explicit tracking or constrained decoding.

Conclusion: The current stack fully addresses the problem of continuous semantic conditioning and autoregressive stabilization. The architecture has proven that a compact adapter layer (~150K parameters) is capable of managing the generation of a frozen 60M model, preserving meaning throughout the entire sequence. Further accuracy improvement (>0.80 overlap) and the elimination of residual mutations require either scaling the base LM (t5-base/large) or introducing an explicit entity tracking/AR cycle consistency mechanism, which is being pursued in a separate research iteration. The current version is established as a working, reproducible baseline for semantic text management tasks.

12. Results of the analysis of the Encode_thoughtV3 model

12.1. Results for the prajjwal1/bert-mini encoder

The results are shown in Tables 5 (val dataset) and 6 (train dataset).

#	Theme Overlap	Original text	Recovered text
1	0.66	Spot. Spot saw the shiny car and said, "Wow, Kitty, your car is so bright and clean!" Kitty smiled and replied, "Thank you, Spot. I polish it every day." After playing with the car, Kitty and Spot felt thirsty. They	saw saw a bright spot. Spot saw Spot and Spot, "Wow, Spot, your home is so bright and clean!" Spot smiled and said, "Thank you, Spot. I ounce it every day." After playing

		found a small pond with clear water. They drank the water and felt very happy. They played together all day and became best friends.	with the car, Spot and Spot felt a deep water. They had a small water filled ice.. They felt happy
2	0.49	Once upon a time, in a big forest, there lived a rhinoceros named Roxy. Roxy loved to climb. She climbed trees, rocks, and hills. One day, Roxy found an icy hill. She had never seen anything like it before. It was shiny and cold, and she wanted to climb it. Roxy tried to climb the icy hill, but it was very slippery. She tried again and again, but she kept falling down. Roxy was sad. She wanted to climb the icy hill so much. Then, she saw a little bird named Billy. Billy saw that Roxy was sad and asked, "Why are you sad, Roxy?" Roxy told Billy about the icy hill and how she couldn't climb it. Billy said, "I have an idea! Let's find some big leaves to put under your feet. They will help you climb the icy hill." Roxy and Billy looked for big leaves and found some. Roxy put the leaves under her feet and tried to climb the icy hill again. This time, Roxy didn't slip. She climbed and climbed until she reached the top of the icy hill. Roxy was so happy! She and Billy played on the icy hill all day. From that day on, Roxy and Billy were the best of friends, and they climbed and played together all the time. And Roxy learned that with a little help from a friend, she could climb anything.	upon a time, there lived a big ecosystem called Spike. Spike loved to climb. She climbclimbed mountains rocks rocks, rocks, and rocks. One day, Spike found an ancient glacier. She never seen anything like it before and cold. It was cold and cold, and she wanted to climb it. Spike tried to climb the cool glacier but but it was very slippery. She tried again and she fell down, but she was still falling. Spike wanted to climb this glacier so much. Then, she saw a little bird named Willow. Lisa saw that Jack was happy
3	0.92	Once upon a time, in a small yard, there was a small daisy. The daisy had a name. Her name was Daisy. Daisy was very small, but she was also very happy. One day, Daisy saw a dog. The dog was big and had a name too. His name was Max. Max liked to play in the yard. Daisy liked to watch Max play. Max and Daisy became friends. Every day, Max would come to the yard to play. Daisy would watch and smile. They were very happy together. And even though Daisy was small, she knew that she had a big friend in Max.	upon a time, in a small yard, there was a large dog. The cat had a name. Her name was Daisy. Daisy was very small, but she was also very happy. One day, Daisy saw a dog. The dog was big and had a name too. His name was Max. Max liked to play in the yard. Max liked to play Max Max Max Max Max Max Max Max Max Max would come to play. Every day, Daisy would go to the yard to play. They smile smile and smile. And even though Tina was quite happy that she knew her little girl
4	0.69	Once upon a time, there was a thoughtful girl named Sue. Sue loved to help her mom around the house. One day, her mom asked her to wipe the table after they ate their lunch. Sue was happy to help. As Sue was wiping the table, she saw a pretty candle on the window sill. The candle was her mom's favorite. Sue wanted to do something nice for her mom, so she said, "Mom, can I light the candle for you?" Her mom said, "Yes, but be very careful." Sue carefully lit the candle and put it on the table. Her mom was so happy to see the pretty candle. They both sat and watched the candle burn. Sue's mom said, "Thank you, Sue, for being so thoughtful and careful." Sue felt proud that she could help her mom. The moral of the story is to always be thoughtful and careful when helping others.	upon upon upon upon a time, there was a kind girl named Sue. Sue loved to help her mom decorate the house around her house. One day, she asked her mom to clean the table and ask them how they ate their lunch. Sue was happy. As Sue was helping to wipe the candle, she saw a large window on the window. The candle was her favorite favorite. Sue wanted to do something nice for her mom, so she said, "Mommy, can you shine the candle for me?" Her mom said, "No, but be careful." They put the candle on the floor

5	0.79	Once upon a time, there was a kind farmer. He had a big cow. The cow was sad. The farmer did not know why. One day, a little boy came to the farm. He saw the sad cow. The boy kneeled down to talk to the cow. "Why are you sad, cow?" he asked. The cow said, "I am lonely. I want a friend." The kind farmer heard the cow. He wanted to help. So, he got another cow to be friends with the sad cow. The sad cow was happy now. They played together every day. And the kind farmer, the little boy, and the two cows all lived happily ever after.	upon upon a time, there was a kind cow. He had a big cow. The cow was sad. The cow did not know why. One day, a little boy came to the farm. The boy saw the sad cow. The boy boy kneed down to talk to the cow. "Why are you sad, sad?" he asked. The cow said, "I am lonely. I want to be friends." The kind cow heard the cow. He wanted to help. So, he got a dog to get fun with the sad cow. The cow was happy happy together. They played

Table 5. Results for the prajjwal1/bert-mini val encoder dataset

#	Theme Overlap	Original text	Recovered text
1	0.75	One day, a little girl named Lily found a needle in her room. She knew it was difficult to play with it because it was sharp. Lily wanted to share the needle with her mom, so she could sew a button on her shirt. Lily went to her mom and said, "Mom, I found this needle. Can you share it with me and sew my shirt?" Her mom smiled and said, "Yes, Lily, we can share the needle and fix your shirt." Together, they shared the needle and sewed the button on Lily's shirt. It was not difficult for them because they were sharing and helping each other. After they finished, Lily thanked her mom for sharing the needle and fixing her shirt. They both felt happy because they had shared and worked together.	a time, Lily found a girl named named in in the room. She knew it was difficult to play with it because it was sharp. Lily wanted to share the needle with her mom, so she could push on a button on her face. Lily went to her mom and said, "Mommy, I find this needle. Can you share it with me?" Her mom smiled and said, "Yes, Lily, we can share the needle and share our match." Together, they shared the needle and pinned the needle for its friend. It was hard to see all of them
2	0.67	Once upon a time, there was a little car named Beep. Beep loved to go fast and play in the sun. Beep was a healthy car because he always had good fuel. Good fuel made Beep happy and strong. One day, Beep was driving in the park when he saw a big tree. The tree had many leaves that were falling. Beep liked how the leaves fall and wanted to play with them. Beep drove under the tree and watched the leaves fall on him. He laughed and beeped his horn. Beep played with the falling leaves all day. When it was time to go home, Beep knew he needed more fuel. He went to the fuel place and got more healthy fuel. Now, Beep was ready to go fast and play again the next day. And Beep lived happily ever after.	upon upon upon upon upon upon upon a time, there was a little bird named Carly. Carly loved to go fast and play in the sun. Carly was a healthy car because it had a consistent fuel. Always good good fuel. Cary was strong and happy. One day, Carly was driving in the park when he saw a big tree. The trees were that that had fallen too. Cary liked how the leaves fell and wanted to play with them. Carly watched under the tree and watched the leaves hang on him. He scouted his octopus
3	0.70	One day, a little fish named Fin was swimming near the shore. He saw a big crab and wanted to be friends. "Hi, I am Fin. Do you want to play?" asked the little fish. The crab looked at Fin and said, "No, I don't want to play. I am cold and I don't feel fine." Fin felt sad but wanted to help the crab feel better. He swam away and thought of a plan. He remembered that the sun could make things warm. So, Fin swam to the top of the water and called to the sun, "Please, sun, help my new friend feel fine and not freeze!" The sun heard Fin's call and shone	a long fish, named Fin was fishing near the shore. He saw a big crab and wanted to be friends. "Hi, I am am you?" asked the crab. The crab looked at the crab and said, "No, I don't want to play. I don't feel sick and I'm feeling cold." Fin felt sad but wanted to help the crab feel better. He listened away and thought of a plan. He remembered that the sun could make things warm. So, Finn swam to the top of

		its warm light on the shore. The crab started to feel better and not so cold. He saw Fin and said, "Thank you, little fish, for making me feel fine. I don't feel like I will freeze now. Let's play together!" And so, Fin and the crab played and became good friends.	the water and called it " "Jored to call back
4	0.67	Once upon a time, in a land full of trees, there was a little cherry tree. The cherry tree was very sad because it did not have any friends. All the other trees were big and strong, but the cherry tree was small and weak. The cherry tree was envious of the big trees. One day, the cherry tree felt a tickle in its branches. It was a little spring wind. The wind told the cherry tree not to be sad. The wind said, "You are special because you have sweet cherries that everyone loves." The cherry tree started to feel a little better. As time went on, the cherry tree grew more and more cherries. All the animals in the land came to eat the cherries and play under the cherry tree. The cherry tree was happy because it had many friends now. The cherry tree learned that being different can be a good thing. And they all lived happily ever after.	a time, in a land of forest, there was a little cherry. The cherry tree was very sad because it did not have any friends. The other trees were big and strong, but the cherry tree was small and bearish. The cherry tree was strappy by the big trees. One day, the cherry tree felt a sparkly leaf in its snow. It was a little wind! The rainy tree told the tree to be sad. The tree said that you are special because you have special gifts that everyone has." The cherry tree started to get better. "Stole down a

Table 6. Results for the prajjwal1/bert-mini encoder train dataset

12.2. Analysis of results

All main experiments were conducted with the frozen prajjwal1/bert-mini base encoder and the t5-small seq2seq decoder adapted via Cross-Attention LoRA. To test the hypothesis of robust autoregressive generation from compressed slots, the following configuration was used: 16 slots × 192 dim, linear projector 192→512, curriculum embedding noise ($\alpha \leq 0.12$), and slot-alignment loss ($\lambda=0.5$). The final number of trainable parameters was ~150,000. Testing was conducted in closed autoregressive (AR) mode on independent val and train datasets.

12.2.1. Autoregressive Cycle Stability and Syntactic Integrity

Unlike V2, where generation collapsed into lexical loops after just 5–10 steps, the current architecture demonstrates complete stability of the closed AR loop throughout the entire target sequence (80–128 tokens). Syntactic breaks, loops on high-frequency tokens (the, and, was), and punctuation degradation are completely eliminated. The model reliably preserves the TinyStories style, grammatical constructions, and logical transitions between sentences. This empirically confirms that the transition to native cross-attention and learning on noisy trajectories (curriculum noise) successfully closes the autoregressive loop without destroying the gradient flow or generalization metrics.

12.2.2. Semantic precision and generalization ability

The key metric for assessing the recovery quality is Theme Overlap—the proportion of semantically significant words from the original preserved in the generated text. On the validation set, the average value was ~0.72, on the training set, ~0.70. The near-complete agreement between the Train and Val metrics confirms a fair generalization: the model does not memorize the dataset, but has learned a stable mapping from "slots to narrative." Early stopping correctly identifies the checkpoint at the peak of validation quality, and there is no discrepancy between the training and inference distributions. In 4 out of 5 test samples, the plot skeleton, causal relationships, and character roles are preserved without structural distortions.

12.2.3. The nature of residual drift and the capacitive limit

Despite syntactic stability and high structural consistency, the generated code exhibits localized semantic mutations. Name and object substitutions (Roxy → Shelley/Spike, cherry → pine, cow → dog) are observed, as well as slight echoes at sentence boundaries (water water, Max Max). Diagnostics show that these artifacts are not the result of overfitting, optimization errors, or insufficient epochs. The cause lies in the fundamental limit of continuous compression: the 16x192 configuration successfully encodes global semantics and narrative outline, but does not have sufficient bandwidth to rigidly capture token-level entities (proper nouns) without an explicit tracking mechanism or constrained decoding. Frozen T5-small language priors further reinforce the tendency to replace rare names with those statistically more probable within the context of the genre.

12.2.4. Conclusions

The current architecture fully addresses the problem of autoregressive stabilization and continuous semantic conditioning. A compact adapter layer (~150K parameters) is capable of managing the generation of a frozen 60M model, preserving meaning, style, and structure throughout the entire sequence. The achieved Theme Overlap ceiling (~0.72) and the nature of residual mutations clearly define the limit of applicability of the current stack: further accuracy improvement (>0.80) and the elimination of entity drift require either scaling the base language model (t5-base/large) or implementing explicit entity tracking/AR cycle-consistency loss. This result is recorded as a reproducible intermediate milestone and a ready-made baseline for the next research iteration.

13. Direction for continuing research and experiments

13.1. Current Limit Diagnosis

The current version of the architecture successfully solved the problem of autoregressive stabilization and continuous semantic conditioning. However, experiments clearly identified a fundamental limitation: the fixed slot matrix (16x192) acts as a global bottleneck. It successfully encodes the plot skeleton and global semantics of short narratives, but lacks sufficient bandwidth to rigidly capture entities, names, and long dependencies. Attempting to compress the entire text into a single static snapshot of meaning contradicts the nature of language, where thought unfolds sequentially and sentences reference each other through coreferences and causal relationships. Further increases in the dimensionality or number of slots within a single matrix yield diminishing returns and do not solve the problem of structural scaling to long texts.

13.2. Concept: from a single matrix to a stream of connected thoughts

The next logical step is to abandon encoding the entire text into a single matrix in favor of a sequence of semantic blocks. The text is broken into fragments (sentences or windows of fixed length), each of which is independently compressed into its own slot matrix using the current SetEncoder + SlotBottleneck mechanism. The result is not a static vector, but a temporal sequence of matrices S_1, S_2, …, S_n, where each matrix encodes the local meaning of a fragment.

The key innovation lies in the inter-block interaction mechanism: matrices must reference each other, similar to how a transformer computes attention between tokens. A lightweight causal transformer over a sequence of slots resolves coreferences, conveys context between sentences, and forms a global narrative structure without a quadratic complexity explosion. The decoder receives not a single projection, but a sliding window of K current and previous matrices, ensuring continuous semantic binding throughout the generation.

13.3. Architectural implications and key challenges

Going to a matrix sequence preserves the current stack as a local module, but adds three critical components:

- Chunker: splits text into semantically complete windows (30–50 tokens) with 20–30% overlap to smooth boundaries and preserve pronominal relationships.

- Inter-Slot Attention: a causal transformer (2-3 layers, dim=192) over a sequence of matrices. Computes inter-slot attention, limited to a sliding window (K=3-4), to keep O(N) complexity and VRAM consumption stable.
- Transition Mechanism: soft interpolation of projections or explicit copying of 2–4 "context slots" from S_{t-1} to S_t to prevent semantic breaks at sentence boundaries.

The main risk lies in training inter-block connections: the model may ignore cross-chunk attention and generate fragments in isolation. To prevent this, InterSlotAlignmentLoss is introduced—a cosine alignment of connected slots of adjacent matrices—as well as curriculum noise at transitions between fragments. Inference requires smooth decoder switching between matrices, which is addressed by a fixed transition_token or soft interpolation of projections at window boundaries.

13.4. Minimum Viable Experiment

To test the hypothesis without rewriting the pipeline, the following protocol is planned:

1. Break TinyStories into sentences or windows of 30–40 tokens.
2. Apply current EncodeThought to each window independently → get a sequence of slot matrices.
3. Add SlotSequenceTransformer (2 layers, causal attention, sliding window K=3) on top of the matrices.
4. Decoder T5 receives projections of the current block + the two previous blocks. Cross-attention is limited by the window.
5. Train end-to-end with the same curriculum noise, adding a light loss to ensure consistency of transitions between blocks.
6. Test on chains of 2–4 sentences. Success metrics: Theme Overlap growth on long dependencies, disappearance of name mutations when transitioning between sentences, and maintenance of syntactic stability.

If the hypothesis is confirmed on short chains, scaling to full stories and long texts becomes an engineering task, not a research one.

13.5. Next Iteration Protocol

We are working on the principle of a one-dimensional experiment: the next version will change only the latent space structuring mechanism, maintaining the current stack (SetEncoder, SlotBottleneck, T5 Cross-LoRA, curriculum noise, alignment loss) as a fixed baseline. The dataset, metrics (Theme Overlap, Entity Retention, AR-Coherence), and validation infrastructure remain fixed. The sole goal is to test whether a sequence of interconnected matrices removes the capacitance ceiling of a fixed bottleneck and eliminates entity drift at fragment boundaries. The next version of the paper will contain exclusively the results of this experiment, its comparative diagnostics with V3, and a final decision on the architecture's readiness for scaling to long narratives.

To be continued …